

AIMS System - Detailed Class Design Documentation

1. Design Class: Product

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	Unique Identifier for the product
2	title	String	null	Name of the product
3	category	String	null	Product category (Book, CD, DVD, or LP record)
4	value	float	0.0	Base price of the product before VAT
5	price	float	0.0	Selling price of the product
6	quantity	int	0	Available quantity in stock
7	description	String	null	Product description
8	imageUrl	String	null	The URL for the image of the product item
9	barCode	String	null	Unique barcode for the product
10	warehouseEntryDate	Date	null	Date when the product entered the warehouse
11	dimensions	String	null	Physical dimensions of the product (e.g., 10x20x5 cm)
12	weight	float	null	Weight of the product in kilograms
13	warehouseProvince	String	null	province of warehouse storing the product
14	warehouseDistrict	String	null	district of warehouse storing the product
15	warehouseAddress	String	null	detailed address of warehouse storing the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	checkProductAvailability	boolean	Checks if the product has sufficient quantity available
2	updateProduct	vold	Updates product information in the system
3	checkEligibilityOfRushDelivery	boolean	Checks if product is eligible for rush delivery
4	createProduct	vold	Create a new product in the system
5	searchProduct	vold	Search for a specific product attributes

Parameter:

- checkProductAvailability(quantity: int): Boolean
 - **quantity:** An integer representing the number of products to check if the available inventory is sufficient.
 - **Return:** true if the available inventory is sufficient, otherwise false.
- updateProduct(productId: int): vold
 - **productId:** productId must exist
- searchProduct(searchCriteria: Map<String, Object>): vold
 - **searchCriteria:** A map containing search criteria (field names and values)

Exception:

- updateProduct(productId: int): vold
 - ProductNotFoundException: If no product with the specified productId is found.
 - ExceedUpdateTimesException: If update more than 30 products a day
 - ExceedUpdatePriceTimesPerDayException: If update price more than twice a day

Method:

How to use parameters/attributes:

- checkProductAvailability(quantity: int): Boolean
 - **Purpose:** This method checks whether the available inventory is sufficient for the requested quantity of a product.
 - **Usage:** If the available stock is at least equal to the requested quantity, the method returns true; otherwise, it returns false.
- updateProduct(productId: int): void
 - **Purpose:** This method updates the details of an existing product based on the provided information.
 - **Usage:** If the product exists, the method updates attributes such as name, price, quantity, etc
- createProduct(): void
 - **Purpose:** This method creates a new product with the provided information.
 - **Usage:** If all required attributes are provided and valid, the product is successfully created and added to the inventory.
- checkEligibilityOfRushDelivery(address: String): Boolean
 - **Purpose:** to check rush delivery support
 - **Usage:** compare 'address' parameter (from delivery information) and address of product (based on information provided by Product Manager)
- searchProduct(searchCriteria: Map<String, Object>): void
 - **Purpose:** Searches for products matching specific criteria.
 - **Usage:** Used to find products based on various attributes.
 - **Processing:**
 1. Constructs a database query based on the search criteria
 2. Executes the query
 3. Returns matching products

State:

- If stock > requested quantity → Available.
- If stock < requested quantity → Not Available.
- If product_Id is invalid → ProductNotFoundException is raised.

2. Design Class: Book

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	author	String	null	Author of the book
2	coverType	String	null	Type of cover (paperback or hardcover)
3	publisher	String	null	Publisher of the book
4	publicationDate	Date	null	Date of publication
5	pages	int	0	Number of pages
6	language	String	null	Language of the book
7	genre	String	null	Genre of the book

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	void	Updates product information in the system
2	createProduct	void	Create a new product in the system

Parameter: None

Exception:

- updateProduct(productId: int): void
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- updateProduct(productId: int): void
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of book.
 - **Usage:** If the book exists, the method updates attributes such as name, price, quantity, etc

- createProduct(): void
 - **Purpose:** Creates a new Book with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the book is successfully created and added to the inventory.

3. Design Class: CD

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	artist	String	null	Name of the artist or band
2	recordLabel	String	null	Name of the record label
3	tracklist	String	null	List of tracks in the CD
4	genre	String	null	Music genre of the CD
5	releaseDate	Date	0	Official release date of the CD

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	void	Updates product information in the system
2	createProduct	void	Create a new product in the system

Parameter: None

Exception:

- updateProduct(productId: int): void
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- updateProduct(productId: int): void
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of CD.
 - **Usage:** If the CD exists, the method updates attributes such as name, price, quantity, etc
- createProduct(): void
 - **Purpose:** Creates a new CD with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the CD is successfully created and added to the inventory.

4. Design Class: DVD

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	discType	String	null	Type of disc (e.g: Blu-ray, HD-DVD)
2	director	String	null	Name of the artist
3	runtime	String	null	Total runtime of the DVD
4	studio	String	null	Name of the studio
5	language	String	null	Language of the content in DVD
5	subtitle	String	null	Language subtitle
6	releaseDate	Date	null	Release date of the DVD
7	genre	String	null	The genre of the DVD

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	void	Updates product information in the system

2	createProduct	vold	Create a new product in the system
---	---------------	------	------------------------------------

Parameter: None

Exception:

- updateProduct(productId: int): vold
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- createProduct():
 - **Purpose:** Creates a new DVD with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the DVD is successfully created and added to the inventory.
- updateProduct(productId: int): vold
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of DVD.
 - **Usage:** If the DVD exists, the method updates attributes such as name, price, quantity, etc

5. Design Class: LP record

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	artist	String	null	Name of the artist or band
2	recordLabel	String	null	Name of the record label
3	tracklist	String	null	List of tracks in the LP record
4	genre	String	null	Music genre of the LP record
5	releaseDate	Date	0	Official release date of the LP record

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	vold	Updates product information in the system
2	createProduct	vold	Create a new product in the system

Parameter: None

Exception:

- updateProduct(productId: int): vold
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- createProduct():
 - **Purpose:** Creates a new DVD with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the DVD is successfully created and added to the inventory.
- updateProduct(productId: int): vold
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of DVD.
 - **Usage:** If the DVD exists, the method updates attributes such as name, price, quantity, etc

6. Design Class: History update

Table 1. Atrribute design

#	Name	Data type	Default value	Description
1	historyId	int	null	Unique Identifier for the product
2	time	Date	null	The time product manager update the product

3	isUpdatePrice	boolean	null	The price of the product is update or not
4	productId	int	null	The Id of the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	recordUpdate	vold	Records the update history of a product when changes are made
2	getUpdateHistory	List<HistoryUpdate>	Retrieves the update history of a product based on productId.
3	getLastUpdate	HistoryUpdate	Retrieves the most recent update of a product.
4	deleteHistory	vold	Deletes the entire update history of a product based on productId.

Parameter:

- recordUpdate(productId: int, isUpdatePrice: boolean): vold
 - productId:** An integer representing the unique Identifier of the updated product.
 - isUpdatePrice:** A boolean indicating whether the product price was updated.
 - Return:** This method does not return a value. It records an update entry in the history log.
- getUpdateHistory(productId: int): List<HistoryUpdate>
 - productId:** An integer representing the unique Identifier of the product.
 - Return:** A list of HistoryUpdate objects containing all update records for the specified product. If no history exists, it returns an empty list.
- getLastUpdate(productId: int): HistoryUpdate
 - productId:** An integer representing the unique Identifier of the product.
 - Return:** A HistoryUpdate object containing the latest update details of the specified product. If no update history exists, it returns null or raises an exception.
- deleteHistory(productId: int): vold
 - productId:** An integer representing the unique Identifier of the product.
 - Return:** This method does not return a value. It deletes all update history records related to the specified product.

Exception:

- recordUpdate(productId: int, isUpdatePrice: boolean): vold
 - ProductNotFoundException: If no product with the specified product_Id is found.
- getUpdateHistory(productId: int): List<HistoryUpdate>
 - ProductNotFoundException: If no product with the specified product_Id is found.
- getLastUpdate(productId: int): HistoryUpdate
 - ProductNotFoundException: If no product with the specified product_Id is found.
- deleteHistory(productId: int): vold
 - ProductNotFoundException: If no product with the specified product_Id is found.

Method:

How to use parameters/attributes:

- recordUpdate(productId: int, isUpdatePrice: boolean): vold
 - Purpose: Records the update history of a product when changes are made.
 - Usage: When a product is updated (either information or price), this method creates a new record with time as the update timestamp and isUpdatePrice indicating whether the price was updated.
- getUpdateHistory(productId: int): List<HistoryUpdate>
 - Purpose: Retrieves the update history of a product based on productId.
 - Usage: If the product has an update history, the method returns a list of previous updates; otherwise, it may return an empty list.
- getLastUpdate(productId: int): HistoryUpdate
 - Purpose: Retrieves the most recent update of a product.
 - Usage: The method finds the latest update record of productId and returns it. If no history exists, it may return null or raise an error.
- deleteHistory(productId: int): vold
 - Purpose: Deletes the entire update history of a product based on productId.
 - Usage: When the update history of a product needs to be removed (e.g., when the product is deleted from the system), this method deletes all related records.

7. Design Class: ViewProductDetailsController

Table 1. Operation design

#	Name	Return type	Description (purpose)
1	RequestToViewProductDetails	vold	Receives a request from the user to view product details based on productId.
2	renderProductDetails	vold	Retrieves product information from

Parameter

- **requestToViewProductDetails(productId: int): void**
 - **productId:** The unique Identifier of the product to view.
- **renderProductDetails(productId: String): void**
 - **productId:** The unique Identifier of the product to view.

Exception

- **requestToViewProductDetails(productId: int): void**
 - **ProductNotFoundException** if the product does not exist in the database.
- **renderProductDetails(productId: String): void**
 - **None**

Method: How to Use Parameters/Attributes

- **RequestToViewProductDetails((productId: int): void**
 - **Purpose:** Sends a request to retrieve product details.
 - **Usage:** If the product exists, its details are displayed on the screen. If the product does not exist, a **ProductNotFoundException** is raised.
- **renderProductDetails**
 - **Purpose:** Retrieves and displays product details.
 - **Usage:** Called internally by requestToViewProductDetails.

State

- If **product exists** → Product details are displayed on the screen.
- If **product does not exist** → **ProductNotFoundException** is raised, and an error message is displayed.

8. Design Class: AddProductToCartController

Table 1. Operation design

#	Name	Return type	Description (purpose)
1	requestToAddProductToCart	vold	Processes request to add product to cart
2	checkProductInCart	boolean	Checks if a specific product is already in the cart.

Parameter:

- **requestToAddProductToCart(productId: String, quantity: int, cartID: int): vold**
 - **productId:** The unique Identifier of the product to add to the cart.
 - **quantity:** The number of units to add.
 - **cartID:** The unique Identifier of the cart to add products.
 - **Return:** None
- **checkProductInCart(productId: String, cartID: Cart): boolean**
 - **productId:** The unique Identifier of the product to check.
 - **cartID:** The unique Identifier of the cart to add products.
 - **Return:** true if the product is present, otherwise false

Exception:

- **requestToAddProductToCart(productId: String, quantity: int, cartID: int): vold**
 - **ProductNotFoundException:** If the specified product does not exist.
 - **InsufficientStockException:** If the requested quantity exceeds available stock.

Method:

How to Use Parameters/Attributes

- **requestToAddProductToCart(productId: String, quantity: int, cartID: int): vold**
 - **Purpose:** This method processes a request to add a product to the shopping cart.
 - **Usage:**
 - If the product exists in inventory and the requested quantity is available, the product is added to the cart, and the inventory stock is reduced accordingly.
 - If the product does not exist, a **ProductNotFoundException** is raised.

- If the requested quantity exceeds available stock, an **InsufficientStockException** is raised.
- **checkProductInCart(productId: String, cartID: Cart): boolean**
 - Purpose: This method checks if a specific product is already in the shopping cart.
 - Usage: Call inslde **requestToAddProductToCart(productId: String, quantity: int, cartID: int): void**
 - Retrieve the product from currentCart using productId.
 - If the product is found, return true.
 - If the product is not found, return false.

State

- If **product exists** and **stock ≥ requested quantity**, product is added to cart, stock is updated.
- If **product exists** but **stock < requested quantity** , **InsufficientStockException** is raised.
- If **productId is invalld**, **ProductNotFoundException** is raised.

9. Design Class: UpdateCartController

Table 1. Example of operation design

#	Name	Return type	Description (purpose)
1	requestToUpdateCart	boolean	Initiates cart update process
2	updateQuantity	void	Updates the quantity of a specific product in the cart.
3	removeProductFromCart	void	Removes a specific product from the cart.

Parameter:

- **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
 - **productId**: The unique Identifier of the product to update in the cart.
 - **cartID**: The shopping cart that contains the products to update quantity.
 - **quantity**: The new quantity of the product.
 - **Return**: None
- **updateQuantity(productId: int, quantity: int, cartID: int): void**
 - **productId**: The unique Identifier of the product whose quantity needs to be updated.
 - **quantity**: The updated quantity to be set for the product.
 - **cartID**: The shopping cart that contains the products to update quantity.
 - **Return**: None
- **removeProductFromCart(productId: int, cartID: int): void**
 - **productId**: The unique Identifier of the product to be removed from the cart.
 - **cartID**: The shopping cart that contains the products to update quantity.

Exception:

- **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
 - **ProductNotFoundException**: If the specified product does not exist.
- **updateQuantity(productId: int, quantity: int, cartID: int): void**
 - **InsufficientStockException**: If the requested quantity exceeds available stock.

Method:

How to Use Parameters/Attributes

- **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
 - **Purpose**: This method initiates the cart update process when a user modifies the quantity or removes an item from the cart.
 - **Usage**:
 - Retrieve the product using productId. If the product is not found, throw ProductNotFoundException.
 - Update the cart.
- **updateQuantity(productId: int, quantity: int, cartID: int): void**
 - **Purpose**: Updates the quantity of a specific product in the shopping cart.
 - **Usage**: Call inslde **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
 - Retrieve the product from currentCart using productId.
 - Valldate the quantity (e.g., ensure it's a positive integer).
 - Update the product quantity in currentCart.If the requested quantity exceeds available stock, throw **InsufficientStockException**
- **removeProductFromCart(productId: int, cartID: int): void**
 - **Purpose**: Removes a product from the shopping cart based on its unique Identifier.
 - **Usage**: Call inslde **requestToUpdateCart(productId: int, quantity: int, cartID: int): void**
 - Retrieve the product from currentCart using productId.
 - Remove the product from currentCart.
 - Update the cart state accordingly.

State:

- **Checking Stock** – Verifying the stock quantity of the product.
- **Available** – The requested quantity is in stock (stock > requested quantity).
- **Not Available** – Insufficient stock (stock < requested quantity).
- **Error: Product Not Found** – The product ID is invalid (ProductNotFoundException).

10. Design Class: ViewCartController

Table 1. operation design

#	Name	Return type	Description (purpose)
1	requestToViewCart	vold	Returns the current cart for viewing

Parameter

- **requestToViewCart(cartID: int): void**
 - **cartID:** The shopping cart ID to be viewed.
 - **Return:** None

Exception

- **requestToViewCart(cartID: int): void**
 - **CartNotFoundException:** If the shopping cart is not found or has not been initialized

Method:

How to Use Parameters/Attributes

- **requestToViewCart(cartID: int): void**
 - **Purpose** This method processes a request to view the shopping cart.
 - **Usage:** If cartID is valid and contains items, its contents are displayed. If the current cart is null or empty, a CartNotFoundException is raised.

State:

- **Checking Cart ID:** The system verifies whether the cart ID is valid.
- **Cart Found:** The cart is available, and its contents are displayed.
- **Cart Not Found:** If the cart does not exist or is uninitialized, a CartNotFoundException is raised.

11. Design Class: Cart

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	cartItemsList	List<CartItem>	empty	List of CartItem in the cart
2	cartId	float	0.0	Unique Identifier for the cart

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	calculateTotalPrice	float	Calculates and returns the total price
2	empty	vold	Empties the cart
3	checkEligibilityOfRushDeliveryOfChosenItems	boolean	Checks if all items are eligible for rush delivery
4	checkAvailabilityOfChosenItems	boolean	Checks if all items have sufficient inventory
5	isEmpty	boolean	Checks if the cart is empty

Parameter:

- **checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean**
 - **province:** The name of the province where the delivery is requested.
 - **district:** The name of the district within the province.
 - **Return:** true if all products are eligible for rush delivery, otherwise false.

Exception: None

Method:

How to use parameters/attributes:

- **calculateTotalPrice(): float**
 - **Purpose:** Computes the total price for the cart.
 - **Usage:** Iterates through all cart items, sums their prices, applies VAT, and returns the total price.
- **empty(): void**
 - **Purpose:** Empties the cart by removing all items.
 - **Usage:** All items are cleared, and subtotal, VAT, and total price are reset to zero.
- **checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean**
 - **Purpose:** Determines if all products in the cart are eligible for rush delivery.
 - **Usage:** Checks each product's delivery eligibility and returns true if all qualify, otherwise false.
- **checkAvailabilityOfChosenItems(province: String, district: String): boolean**
 - **Purpose:** Checks if all items in the cart have sufficient stock.
 - **Usage:** Iterates through the chosen cart items and verifies stock availability. Returns true if all items are available, otherwise false.
- **isEmpty(): boolean**
 - **Purpose:** Checks if the cart is empty.
 - **Usage:** Returns true if there are no items in the cart, otherwise false.

State:

- If cartItemsList is empty → Empty
- If cartItemsList contains at least one item → Active
- If checkout() is called and payment is not completed → CheckedOut
- If payment is successful → Pald
- If cart is emptied (either manually or after order completion) → Empty

12. Class Design: CartItem

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	Id of the product in the cart
2	quantity	int	0	Quantity of the product
3	cartId	int	null	Unique Identifier for the cart
4	cartItemId	int	null	Unique Identifier for the cart item

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	changeQuantityOfProducts.	void	Updates the quantity and recalculates subtotal
2	addProductToCart	void	Adds a product to the cart with specified quantity

Parameter:

- **addProductToCart(product: Product, quantity: int): voids**
 - **product:** A Product object representing the product to add.
 - **quantity:** An integer representing the number of products to add.
 - **Return:** None.
- **changeQuantityofProduct(product: Product, quantity: int): void**
 - **product:** A Product object representing the product to update.
 - **quantity:** An integer representing the new quantity.
 - **Return:** None.

Exception:

- **addProductToCart(productId: int, quantity: int): void**
 - **ProductNotFoundException:** Raised if the specified product does not exist in the system.
 - **InsufficientStockException:** Raised if the requested quantity exceeds the available stock.
- **changeQuantityOfProduct(productId: int, quantity: int): void**
 - **ProductNotFoundException:** Raised if the product is not found in the cart.
 - **InsufficientStockException:** Raised if the requested quantity exceeds the available stock.

Method:

How to use parameters/attributes:

- **addProductToCart(ProductId: int):**
 - **Purpose:** Adds a product to the cart with a specified quantity if stock is sufficient.
 - **Usage:** If the product exists and the requested quantity is available, it is added to the cart. Otherwise, InsufficientStockException is raised.
- **changeQuantityofProduct(ProductId: int):**
 - **Purpose:** Updates the quantity of an existing product in the cart.
 - **Usage:** If the product is already in the cart and the stock allows, the quantity is updated. Otherwise, InsufficientStockException is raised.

State

- **Available** :If stock > requested quantity, the product is available for purchase.
- **Not Available** :If stock < requested quantity, the requested quantity cannot be fulfilled.
- **Product Not Found**:If product_Id is invalld (i.e., the product does not exist in the system), a ProductNotFoundException is raised

13. Design Class: PaymentTransaction

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	int	null	Unique Identifier for the transaction
2	amount	float	0.0	Payment amount
3	paymentMethod	String	"Credit Card"	Method of payment
4	transactionDate	Date	current date	Date of transaction
5	status	String	"Pending"	Transaction status (Pending, Success, Failed, Refunded)
6	content	String	null	Transaction details or description
7	orderId	int	null	Id of the associated order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	evalutePaymentResult	boolean	Processes the payment result from VNPay
2	save	vold	Saves the transaction to the database
3	transactionInfo	String	Returns formatted transaction information
4	saveRefundTransaction	vold	Saves refund transaction information
5	createTransaction	vold	Creates (stores) a new payment transaction in the system

Parameter:

- transactionInfo(transactionId: String) : PaymentTransaction
 - transactionId: An integer representing the unique Id of the transaction.
 - Return: A PaymentTransaction object containing the transaction details.

Exception:

- TransactionNotFoundException if referencing a non-existent transaction.
- InvalldTransactionAmountException – If the amount is invalld (e.g., negative or zero when it shouldn’t be).

Method:

- save()
 - Purpose: Commits the current transaction state to persistent storage.
 - Usage: Called after creating or updating any transaction details (e.g., setting the status to “completed”).
- transactionInfo(transactionId: String): PaymentTransaction
 - Purpose: Retrieves the details of a specific payment transaction by its unique Id.
 - Usage: If transactionId exists, returns a PaymentTransaction object. If it does not exist, raises TransactionNotFoundException.

State:

- If transactionId is invalld → TransactionNotFoundException is raised.
- If transactionStatus is "completed" → Payment is successful and no further changes unless a refund is requested.
- If processRefund is successful → transactionStatus might be changed to “refunded”, and isRefunded is set to true.
- refundTransactionId may be generated or updated when a refund is initiated.

14. Design Class: PlaceOrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentCart	Cart	null	Reference to the customer's current shopping cart
2	currentOrder	Order	null	Order being created in the current process
3	deliveryInfo	DeliveryInfo	null	Delivery information for the current order
4	currentInvoice	Invoice	null	Invoice generated for the current order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	requestToPlaceOrder	boolean	Initiates the order placement process with items from cart
2	checkAvailabilityOfChosenItems	boolean	Verifies if all items in cart are available in sufficient quantity
3	Order	Order	Creates a new order with items from the cart
4	DeliveryForm	DeliveryForm	Creates and returns a delivery form for customer input
5	submitDeliveryInfo	boolean	Processes the delivery information submitted by customer
6	checkValldityOfDeliveryInfo	boolean	Valldates the delivery information format and completeness
7	DeliveryInfo	DeliveryInfo	Creates a delivery info object from valldated input
8	calculateShippingFee	float	Calculates shipping fee based on product weight and delivery location
9	Invoice	Invoice	Creates an invoice with order and delivery information
10	placeOrder	boolean	Finalizes the order placement after payment is processed
11	save	vold	Saves the confirmed order to the database
12	empty	vold	Clears the cart after successful order placement
13	sendSuccessOrderEmail	boolean	Sends order confirmation email to customer
14	OrderInfoScreen	OrderInfoScre en	Creates a screen displaying order and transaction details
15	placeRushOrder	boolean	Verify if there exists any products for rush delivery from cart

Parameter:

- **requestToPlaceOrder(Cart): boolean**
 - **cartObj:** The shopping cart containing products to be ordered
 - **Return:** true if all products are available and order can proceed, false otherwise
- **checkAvailabilityOfChosenItems(Cart): boolean**
 - **cart:** The shopping cart to check availability for
 - **Return:** true if all items are available in sufficient quantity, false otherwise
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **deliveryInfo:** Map containing delivery information fields (name, address, etc.)
 - **Return:** true if delivery information is valld, false otherwise

- **checkValldityOfDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **deliveryInfo:** Map containing delivery information to valldate
 - **Return:** true if all required fields are valld, false otherwise
- **calculateShippingFee(deliveryInfo: DeliveryInfo, order: Order): float**
 - **deliveryInfo:** Customer's delivery information including location
 - **order:** The order containing products with weight information
 - **Return:** The calculated shipping fee in the local currency
- **placeOrder(transactionInfo: Map<String, String>): boolean**
 - **transactionInfo:** Payment transaction details
 - **Return:** true if order was successfully placed, false otherwise
- **placeRushOrder(province: String, district: String): boolean**
 - **province:** entered by a Administrator
 - **district:** entered by a Administrator
 - **Return:** true if there exists any cart items for rush delivery

Exception:

- **requestToPlaceOrder(Cart): boolean**
 - **EmptyCartException:** Thrown if cart is empty
 - **InvalidCartException:** Thrown if cart contains invalld items
- **checkAvailabilityOfChosenItems(Cart): boolean**
 - **DatabaseConnectionException:** Thrown if unable to check product availability
 - **InsufficientStockException:** Details which products have insufficient stock
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **InvalidDeliveryInfoException:** Details which fields are invalld or missing
- **requestToPlaceOrder(Cart): boolean**
 - **UnsupportedLocationException:** Thrown if delivery location is not supported
 - **InvalidWeightException:** Thrown if product weight data is missing or invalld

Method:

- **requestToPlaceOrder(cartObj: Cart): boolean**
 - **Purpose:** Initiates the order placement process by checking product availability
 - **Processing:**
 - Valldates that the cart is not empty
 - For each item in the cart:
 - Calls checkProductAvailability on the Product object
 - If any product is unavailable, returns false and provldes details on unavailable items
 - If all products are available, creates a new Order object
 - Creates and returns a DeliveryForm for customer input
 - **Return:** true if order process can continue, false if any products are unavailable
- **submitDeliveryInfo(deliveryInfo: Map<String, String>): boolean**
 - **Purpose:** Processes and valldates delivery information provided by customer
 - **Processing:**
 - Valldates all required delivery information fields
 - Creates a DeliveryInfo object with the valldated information
 - Calculates shipping fee based on product weights and delivery location
 - Creates an Invoice with order details, products, VAT, and shipping fee
 - Returns an InvoiceScreen for customer review
 - **Return:** true if delivery information is valld and processing continues, false otherwise
- **placeOrder(transactionInfo: Map<String, String>): boolean**
 - **Purpose:** Finalizes order placement after successful payment
 - **Processing:**
 - Saves the order to the database with "Pending Processing" status
 - Empties the cart
 - Sends order confirmation email to customer
 - Creates and displays OrderInfoScreen with order and transaction details
 - **Return:** true if order was successfully placed, false if any step fails

State:

- Before requestToPlaceOrder is called, currentCart contains the customer's selected products
- After successful requestToPlaceOrder, currentOrder contains a new Order object
- After submitDeliveryInfo, deliveryInfo contains valldated delivery information and currentInvoice contains the generated invoice
- After successful placeOrder, the order is saved to the database, the cart is emptied, and confirmation email is sent

15. Design Class: SearchController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productList	List<Product>	empty list	List of products in the system
2	searchResults	List<Product>	Empty list	Products matching search
3	maxResultsPerPage	int	20	Maximum number of products displayed per page

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	searchKeyWord	List<Product>	Search for products based on keyword
2	searchProduct	List<Product>	Perform search within product list

Parameter:

- **searchKeyWord(keyword: String): List<Product>**
 - **keyword:** String keyword input from HomeScreen
 - **Return:** List of products matching the keyword
- **searchProduct(keyword: String): List<Product>**
 - **keyword:** String keyword to compare with product attributes
 - **Return:** List of products matching the keyword

Exception:

- **searchKeyWord(keyword: String): List<Product>**
 - **EmptyKeywordException:** Thrown if search keyword is empty or null

Method:

- **searchKeyWord(keyword: String): List<Product>**
 - **Purpose:** This method receives keywords from HomeScreen and processes the search
 - **Processing:**
 1. Valldate keyword
 2. Call searchProduct method with provlded keyword
 3. Return search results limited by maxResultsPerPage
 - **Return:** List of products matching the keyword, limited to 20 products per page
- **searchProduct(keyword: String): List<Product>**
 - **Purpose:** Perform product search based on keyword
 - **Processing:**
 1. Iterate through the product list
 2. Compare keyword with product attributes (name, description, type, etc.)
 3. Collect matching products
 - **Return:** List of all products matching the keyword

State:

- When no search has been performed, searchResults is an empty list
- After search, searchResults contains products matching the keyword
- If no matching products are found, searchResults is an empty list

16. Design Class: Invoice

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	invoiceId	int	null	Unique Identifier for the invoice
2	order	Order	null	Order associated with this invoice
3	issueDate	Date	current date	Date when invoice was created
4	productList	List<OrderItem >	empty list	List of products in the invoice
5	subtotal	float	0.0	Total price excluding VAT
6	VAT	float	0.0	Value Added Tax amount (10% of subtotal)
7	totalWithVAT	float	0.0	Subtotal + VAT

8	regularDeliveryFee	float	0.0	Fee for regular delivery items
9	rushDeliveryFee	float	0.0	Fee for rush delivery items
10	totalAmount	float	0.0	Final amount to be paid
11	paymentMethod	String	"Credit Card"	Method of payment
12	recipientInfo	DeliveryInfo	null	Delivery information of the recipient
13	VAT_RATE	float	0.1	Value-added tax rate (constant 10%)

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	createInvoice	Invoice	Creates a new invoice with order and delivery information
2	calculateSubtotal	float	Calculates the subtotal price of all products
3	calculateVAT	float	Calculates the VAT amount based on subtotal
4	calculateTotalWithVAT	float	Calculates the total amount including VAT
5	calculateDeliveryFees	vold	Calculates separate regular and rush delivery fees
6	calculateTotalAmount	float	Calculates the final total amount to be paid
7	generateInvoiceDetails	Map<String, Object>	Generates a complete map of invoice details
8	save	vold	Saves the invoice to the system
9	getInvoiceById	Invoice	Retrieves an invoice by its Id
10	Invoice	vold	Constructor for creating an invoice

Parameter:

- **createInvoice(cart: Cart) : Invoice**
 - cart: The user’s cart data.
 - Return: A fully built Invoice with items, total amounts, and an assigned Id.

Exception:

- InvoiceSaveException – Thrown if saving the invoice fails (database error, disk error, etc.).
- InvalidCartException – If cart is empty or invalid when calling createInvoice().

Method:

- Invoice() (constructor)
 - Purpose: Sets up default values (e.g., status = "unpaid").
 - Usage: Called internally or externally to create an invoice instance.
- createInvoice(cart: Cart): Invoice
 - Purpose: Takes a cart object, calculates subtotal, VAT, and total, sets invoice items, then returns an Invoice.
 - Usage: Typically called by InvoiceScreen after the user finalizes the cart.
- save()
 - Purpose: Persists the invoice data (items, amounts, status) into a database or storage.
 - Usage: Called once the invoice is ready to be stored or updated (e.g., after changing status).

State

- status transitions: “unpaid” → “paid” → possibly “cancelled.”
- totalAmountInclVAT = totalAmountExclVAT * 1.1 (assuming 10% VAT).
- finalAmount = totalAmountInclVAT + deliveryFee.

17. Design Class: DeliveryInfo

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	recipientName	String	null	Name of the recipient
2	email	String	null	Email address
3	phoneNumber	String	null	Phone number
4	province	String	null	Province/city
5	district	String	null	District
6	address	String	null	Delivery address
7	deliveryMethod	String	"Regular"	Delivery method (Regular/Rush)
8	deliveryTime	String	0.0	Requested time for rush delivery
9	isRushDeliveryEligible	boolean	false	Indicates if address is eligible for rush delivery
10	deliveryInstructions	String	null	Special delivery instructions

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DeliveryInfo	vold	Constructor for creating delivery information
2	valldateDeliveryInfo	boolean	Valldates all required delivery information fields
3	isEligibleForRushDelivery	boolean	Calculates shipping fee based on product weight and location
4	calculateShippingFee	float	Calculates shipping fee based on product weight and location
5	calculateRushDeliveryFee	float	Calculates rush delivery fee for eligible products
6	getDeliveryInfo	Map<String, String>	Returns all delivery information as a map
7	updateDeliveryInfo	boolean	Updates the delivery information with new values
8	isInHanoiInnerCity	boolean	Checks if delivery is to inner Hanoi districts

Parameter:

- **DeliveryInfo(recipientName: String, email: String, phoneNumber: String, province: String, district: String, address: String, deliveryMethod: String)**
 - **recipientName:** Full name of the recipient
 - **email:** Contact email address
 - **phoneNumber:** Contact phone number
 - **province:** Delivery province/city
 - **district:** Delivery district
 - **address:** Detailed delivery address
 - **deliveryMethod:** Delivery method (Regular or Rush)
- **valldateDeliveryInfo(): boolean**
 - **Return:** true if all required fields are valld, false otherwise
- **isEligibleForRushDelivery(): boolean**
 - **Return:** true if the address is in an area that supports rush delivery, false otherwise
- **calculateShippingFee(weight: float): float**
 - **weight:** Total weight of the products in kilograms
 - **Return:** The calculated shipping fee in VND
- **calculateRushDeliveryFee(numItems: int): float**
 - **numItems:** Number of items for rush delivery
 - **Return:** The calculated rush delivery fee in VND
- **getDeliveryInfo(): Map<String, String>**
 - **Return:** Map containing all delivery information fields
- **updateDeliveryInfo(updatedInfo: Map<String, String>): boolean**
 - **updatedInfo:** Map containing updated delivery information fields
 - **Return:** true if update was successful, false otherwise
- **isInHanoiInnerCity(): boolean**
 - **Return:** true if the delivery district is in inner Hanoi city, false otherwise

Exception:

- **valldateDeliveryInfo(): boolean**
 - **InvalidEmailException:** Thrown if email format is invalld

- **InvalidPhoneNumberException**: Thrown if phone number format is invalid
 - **MissingRequiredFieldException**: Thrown if any required field is missing
- **calculateShippingFee(weight: float): float**
 - **InvalidWeightException**: Thrown if weight is negative or zero
 - **UnsupportedLocationException**: Thrown if delivery location is not supported

Method:

- **validateDeliveryInfo(): boolean**
 - **Purpose**: Validates all required delivery information fields for completeness and format
 - **Processing**:
 1. Checks that recipient name is not empty
 2. Validates email format using regex pattern
 3. Validates phone number format using regex pattern
 4. Checks that province, district, and address are not empty
 5. If delivery method is "Rush", checks that the address is eligible for rush delivery
 - **Return**: true if all validations pass, false otherwise with appropriate error messages
- **isEligibleForRushDelivery(): boolean**
 - **Purpose**: Determines if the delivery address is eligible for rush delivery
 - **Processing**:
 1. Checks if province is "Hanoi"
 2. Checks if district is one of the inner city districts of Hanoi
 - **Return**: true if in eligible area, false otherwise
- **calculateShippingFee(weight: float): float**
 - **Purpose**: Calculates the shipping fee based on product weight and delivery location
 - **Processing**:
 1. Checks if delivery is to inner city Hanoi or Ho Chi Minh City
 - If yes, base fee is 22,000 VND for first 3kg
 - If no, base fee is 30,000 VND for first 0.5kg
 2. Adds 2,500 VND for every additional 0.5kg
 3. Applies free shipping if order value exceeds 100,000 VND (up to 25,000 VND)
 - **Return**: The calculated shipping fee
- **calculateRushDeliveryFee(numItems: int): float**
 - **Purpose**: Calculates additional fee for rush delivery items
 - **Processing**:
 1. Multiplies number of rush delivery items by 10,000 VND
 2. Adds this to the base shipping fee
 - **Return**: The calculated rush delivery fee

State:

- After initialization, all attributes are set to their default values or to the values provided in the constructor
- After validation, the object represents a validated delivery information record
- If rush delivery is requested, deliveryTime and deliveryInstructions are set, and isRushDeliveryEligible flag is checked

18. Design Class: OrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentOrder	Order	null	Currently selected order
2	orderList	List<Order>	empty	A list of orders loaded for viewing or processing
3	alreadyProcessed	boolean	false	Indicates if an order has already been processed or canceled

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	approveOrder	vold	Approves a pending order
2	rejectOrder	vold	Rejects a pending order with reason
3	viewOrderList	vold	Retrieves or displays a list of pending orders.
4	getOrderDetails	vold	Gets details for a specific order
5	checkAvailableInventory	boolean	Checks if there's enough inventory for an order
6	processCancel	boolean	Processes the cancellation of an order

7	createOrder	Order	Creates a new order in the system
8	refundConfirm	vold	Initiates or confirms a refund operation via VNPay.
9	selectCancel	boolean	Handles user requests to cancel the order.
10	checkAlreadyProcessed	boolean	Checks if the current order is already processed (approved/rejected).
11	getOrder	Order	Returns the current Order object in the controller

Parameter.

- checkAlreadyProcessed(orderId: String): boolean
 - orderId: The Id of the order to check
 - Return: true if the order is already approved or rejected, otherwise false.
- selectCancel(orderId: String): boolean
 - orderId: The order to attempt to cancel.
 - Return: true if cancellation is selected, false otherwise.
- refundConfirm(orderId: String): vold
 - orderId: The order to be refunded.
 - processCancel(orderId: String): boolean
- ApproveOrder(orderId: String): vold
 - orderId: The Id of the order to approve
- RejectOrder(orderId: String): vold
 - orderId: The Id of the order to reject
- processCancel(orderId: String): boolean
 - orderId: The Id of the order to finalize cancellation for.
 - Return: true if successfully canceled, otherwise false.
- GetOrderDetail(orderId: String)
 - orderId: The Id of the order whose details are to be fetched.
 - Return: detail information

Exception

- OrderNotFoundException: If the specified orderId does not exist.
- OrderAlreadyApprovedException: If trying to approve/reject/cancel an order that is already processed.

Method

- approveOrder(orderId)
 - Purpose: Set order status to "approved" and decrement inventory.
 - Usage: Called by product manager or automated system upon reviewing an order.
- rejectOrder(orderId)
 - Purpose: Set order status to "rejected". Possibly triggers a refund if payment was collected.
- checkAlreadyProcessed(orderId)
 - Purpose: Quickly verify whether an order is no longer in "pending" state.
- selectCancel(orderId) and processCancel(orderId)
 - Purpose: used in two-step cancellation processes.

State

- If orderStatus is approved or rejected → The order is processed, thus cannot be changed.
- If product is insufficient → approveOrder might either reject the order or throw an exception.

19. Design Class: User

Description: ‘Model’ component containing user information as its attributes, using setter/getter operations to create/update or retrieve its attribute. This model is mapped to an equivalent database table.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userId	Integer	null	- randomly auto-generated - exclusive
2	username	String	null	exclusively exists to Identify users using the software.
3	email	String	null	saved when using gmail for registration
4	password	String	null	must be hashed before stored into database
5	role	String	Product Manager	Administrator / Product Manager
6	status	Boolean	True	True as Unblocked, False as Blocked

Table 2. Operation design

#	Name	Return type	Description
1	getUserId	Integer	retrieve ‘Id’ attribute
2	getUsername	String	retrieve ‘username’ attribute
3	getEmail	String	retrieve ‘email’ attribute
4	getPassword	String	retrieve ‘password’ attribute
5	getRole	String	retrieve ‘role’ attribute
6	getStatus	Boolean	update ‘status’ attribute
7	User	constructor	constructor with all mandatory attributes

Parameter:

- **setter operations** have only a parameter with data type as the corresponding attribute’s data type.
- **getter operations** have no parameters.
- **User(userId: Integer, username: String, email: String, password: String, role: String, status: Boolean)**

Exception

- **all setter operations** can throw a **MandatoryFieldException** which avoids blanking fields.

Method:

- **setter methods** are implemented by assigning the corresponding attribute to a specific value.
- **getter methods** are implemented by returning the value of the corresponding attribute.

State:

- if any parameter in **setter operations** is blank, the method can throw ‘**MandatoryFieldException**’.

20. Design Class: UserService

Description: Service component managing user information in related-business logic.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	users	List<User>	null	list of all users from database
2	currentUser	User	null	current user

Table 2. Example of operation design

#	Name	Return	Description
1	getAllUsers	List<User>	fetch all User instances
2	getUserById	User	fetch User instance by ‘userId’
3	getUserByUsername	User	fetch User instance by ‘username’
4	checkExistenceOfUsername	Boolean	check if the provlded username exists in AIMS system
5	checkSecurityOfPassword	Boolean	check constraints of password to secure password.
6	checkValidityOfEmail	Boolean	check if the provlded email address exists.
6	saveUser	vold	save a specific user into database

Parameter:

- getAllUser(): vold
 - **Return:** true if the available inventory is sufficient, otherwise false.
- getUserById(userId: Integer): User
 - **userId:** An integer representing the unique Identifier of the user.
 - **Return:** a User instance with a user Id equaling the given parameter.
- checkExistenceOfUsername(username: String): Boolean
 - **username:** a String representing username of the user, entered when create/update user procedures
 - **Return:** return true if username exists on database and false if username does not exist.
- checkSecurityOfPassword(password: String): Boolean

- **password:** a String representing the password of the user entered when create/update user procedures, need to use the hash algorithm before comparison.
 - **Return:** return true if password matches security constraints, false if not.
- checkExistenceOfEmail(email: String): Boolean
 - **password:** a String representing the email of the user entered when create/update user procedures
 - **Return:** return true if email exists, false if not.
- saveUser(user: User): void
 - **user:** an User instance which contains user information provided by ‘Administrator.
 - **Return:** void.

Method

- getAllUser(): void
 - **Purpose:** to fetch all users to update its attribute ‘users’ after each time database updates.
 - **Usage:** connect to database then use fetch command of used DBMS.
- getUserById(usedId: Integer): User
 - **Purpose:** to fetch a specific User which is used to display and valldate information.
 - **Usage:** connect to DBMS before executing the retrieve command.
- checkIfUsernameExists(username: String): Boolean
 - **Purpose:** used in login, create or update with user information.
 - **Usage:** compare with information contained in User instance.
- checkSecurityOfPassword(password: String): Boolean
 - **Purpose:** secure password
 - **Usage:** use several standard password constraints.
- saveUser(user: User): void
 - **Purpose:** used in creating/ updating user information.
 - **Usage:** connect to DBMS before executing update command.

Exception:

- getUserById, getUserByUsername: throwing **UserNotFoundException** if there is no User with the given ‘userId’.

State: no specific information.

21. Design Class: UserController

Description: Controller component managing UI display following the related events.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userService	UserService	null	- an interface provlding services to manage related-business logic

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	submitUserInformation	vold	- triggered when a user clicks Button to finish entering a user information form.
2	valldateUserInformation	Boolean	- subroutine of ‘submitUserInformation’ - valldate user information - display notification on invalld fields
3	notifyUserViaEmail	boolean	- subroutine of ‘submitUserInformation’ - send email notification to users via EmailManager’s services.
4	changeAccountStatus	vold	- triggered when a user toggle the status Button on a specific user

Parameter:

- **submitUserInformation(user: User):**
 - **user:** A User instance containing new user information.
 - **Return:** vold.
- **valldateUserInformation(user: User): Boolean**
 - **user:** A User instance containing new user information.
 - **Return:** true if user information is valldated, otherwise, notify user to enter again.
- **notifyUserViaEmail(user: User): vold**
 - **user:** A User instance containing new user information.
 - **Return:** vold.
- **changeAccountStatus(user: User): vold**
 - **user:** A User instance containing new user information.

- **Return:** void.

Exception: no specific information

Method:

- **submitUserInformation(user: User): void**
 - **Purpose:** wrap all subroutines from verification and notification after successful creation/ update.
 - **Usage:** called when a user finishes entering user information form
- **valldateUserInformation(user: User): Boolean**
 - **Purpose:** verify all provded information
 - **Usage:** using checking services from its ‘UserService’ attribute.
- **notifyUserViaEmail(user: User): void**
 - **Purpose:** provlde user information to log into the AIMS system.
 - **Usage:** using EmailManager to send a notification to a user.
- **changeAccountStatus(): void**
 - **Purpose:** modify status of a user’s account from blocked to unblocked and from unblocked to blocked.
 - **Usage:** be called when you toggle Button ‘status’

State:

- **submitUserInformation(user: User): void**
 - if ‘valldateUserInformation’ succeeds, go through and ‘notifyUserViaEmail’.

22. Design Class: DashboardView

Description: View component managing UI components and listening to events to perform action provided by Controller component.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	tableView	javafx.scene.control.tableView<User>	null	display a list of all users
2	root	javafx.scene.layout.BorderPane	null	root UI component
3	userController	UserController	null	corresponding Controller

Table 2. Example of operation design

#	Name	Return	Description
1	createTopNav	void	create navigation bar on the top of dashboard interface
2	createSlideNav	void	create navigation bar on the left slde of dashboard interface
3	createUserTableView	void	create a table containing a list of all users.
4	createUserInformationForm	void	create a modal display a user information form

Parameter:

- **createTopNav(): void**
- **createSlideNav(): void**
- **createUserTableView(): void**
- **createUserInformationForm(user: User): void**

Exception: no specific information

Method:

- **createTopNav(): void**
 - **Purpose:** initialize top navigation bar including several options.
 - **Usage:** called inslde class constructor.
- **createSlideNav(): void**
 - **Purpose:** initialize slde navigation bar.
 - **Usage:** called inslde class constructor.
- **createUserTableView(): void**
 - **Purpose:** create a recyclable stage to navigate between multiple contents.
 - **Usage:** called inslde class constructor.
- **createUserInformationForm(user: User): void**
 - **Purpose:** create UI component to display user information form require user to enter all mandatory information.
 - **Usage:** be called when a user clicks Button to create a new user or update a specific user.

State: no specific information

23. Design Class: UserInformationForm

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	saveButton	javafx.scene.control.Button	null	button to save entered user information
2	statusSwitch	javafx.scene.control.RadioButton	null	switch used to modify account status
2	labels	List<javafx.scene.control.Label>	null	“Username”, “Password”, ...
3	textFields	List<javafx.scene.control.TextField>	null	input components

Table 2 . Operation design

#	Name	Return	Description
1	UserInformationForm	constructor	Constructor which initializes resource to render and handle UserInformationForm modal

Parameter:

- **UserInformationForm(user: User, userController: UserController)**

Exception: no specific information

Method:

- **UserInformationForm(user: User, userController: UserController)**
 - **Purpose:** initialize a form to edit on.
 - **Usage:** called when you click edit/ create button on ‘DashboardView’ component.

State: no specific information

24. Design Class: DeliveryForm

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	checkoutButton	javafx.scene.con trol.Button	null	button to save entered user information
2	labels	List<javafx.scen e.control.Label>	null	“Username”, “Password”, ...
3	textFields	List<javafx.scen e.control.TextFiel d>	null	input components
4	placeOrderController	PlaceOrderContr oller	null	Controller managing rush delivery condition

Table 2 . Operation design

#	Name	Return	Description
1	DeliveryForm	constructor	Constructor which initializes resource to render and handle DeliveryForm modal
2	addAdditionalDeliv eryInformation	vold	display more field when Customer select rush delivery as delivery method

Parameter:

- DeliveryForm(user: User, placeOrderController: PlaceOrderController)
- addAdditionalDeliveryInformation(): void

Exception: no specific information

Method:

- `UserInformationForm(user: User, userController: UserController)`
 - **Purpose:** initialize a form to edit on.
 - **Usage:** called when you click edit/ create button on ‘DashboardView’ component.
- `addAdditionalDeliveryInformation(): void`
 - **Purpose:** display more field when Customer select rush delivery as delivery method
 - **Usage:** called when Customer selects rush delivery and there exists some items for rush delivery.

State: no specific information

25. Design Class: DatabaseManager

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	URL	String	null	url to connect to database system
2	USERNAME	String	null	username to connect to database system
3	PASSWORD	String	null	password to connect database system

Table 2 . Operation design

#	Name	Return	Description
1	getConnection	java.sql.Connection	get connection before command execution

Parameter:

- `getConnection(): java.sql.Connection`

Exception:

- `getConnection(): java.sql.Connection`
 - throw java.sql.SQLException when can not connect to the database system, ensure configuration information and database server activated.

Method:

- `getConnection(): java.sql.Connection`
 - **Purpose:** used to establish connection between the AIMS application with the database system.
 - **Usage:** called when you need to execute a command in the database system.

State:

- `getConnection(): java.sql.Connection`
 - using ‘try-catch’ mechanism to avoid system corruption.

26. Design Class: EmailManager

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	STMPHost	String	null	SMTP host
2	STMPPort	String	null	port to send email
3	senderEmail	String	null	email address of sender
4	password	String	null	password equivalent email address of sender

Table 2 . Operation design

#	Name	Return	Description
1	isEmailValld	Boolean	check email format

2	sendEmail	Boolean	send email by this method
---	-----------	---------	---------------------------

Parameter:

- isEmailValld(email: String): Boolean
 - **email**: email address of sender
 - **Return**: true if email format is matched, otherwise false
- send(recipient: String, subject: String, messageBody: String): void
 - **email**: email address of sender
 - **Return**: void

Exception:

- **send(recipient: String, subject: String, messageBody: String): void**
 - throw jakarta.mail.MessagingException when a user can not send email.

Method:

- **send(recipient: String, subject: String, messageBody: String): void**
 - **Purpose**: verify email address and send success confirmation when creating/ updating user information.
 - **Usage**: called when notifying the updated user or created user from Controller components.

State:

- **send(recipient: String, subject: String, messageBody: String): void**
 - using ‘isEmailValld’ to check before STMP connection establishment
 - using ‘try-catch’ mechanism to avoid system corruption.

27. Design Class: Order

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	Int	"Pending"	Unique Identifier for the order
2	status	String	empty list	Current status: "pending," "approved," "rejected," "canceled," etc.
3	orderItems	List<OrderItem>	empty list	Collection of items (products + quantity) in this order
4	rushOrderItems	List<OrderItem>	empty list	Collection of items eligible for rush delivery
5	regularOrderItems	List<OrderItem>	empty list	Collection of items for regular delivery
6	createdDate	DateTime	current time	Timestamp when the order is first created
7	subtotal	float	0.0	Sum of all product prices excluding VAT
8	vatAmount	float	0.0	Calculated VAT amount (10% of subtotal)
9	regularDeliveryFee	float	0.0	Shipping fee for regular delivery items
10	rushDeliveryFee	float	0.0	Shipping fee for rush delivery items
11	totalAmount	float	0.0	Total amount including VAT and all delivery fees
12	deliveryInfo	DeliveryInfo	null	Delivery information associated with this order
13	paymentTransactionId	int	null	Reference to payment transaction
14	VAT_RATE	float	0.1	Value-added tax rate (constant 10%)

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Order()	void	Constructor for creating a new Order object
2	updateToReject()	void	Sets the order's status to "rejected"
3	updateToApprove()	void	Sets the order's status to "approved"
4	displayApproveStatus()	void	Shows or logs that the order has been approved
5	getOrderDetails	Order	Retrieves/shows the order's item list, price, etc.
6	updateStatus()	void	Updates this order's status based on input
7	save()	void	Persists this order in storage
8	calculateSubtotal	float	Calculates the sum of all product prices excluding VAT

9	calculateVAT	float	Calculates the VAT amount based on subtotal
10	calculateRegularDeliveryFee	float	Calculates shipping fee for regular delivery items
11	calculateRushDeliveryFee	float	Calculates shipping fee for rush delivery items
12	calculateTotalAmount	float	Calculates the total amount including VAT and all delivery fees
13	isEligibleForFreeShipping	boolean	Checks if order is eligible for free regular shipping
14	createOrderListScreen	float	Creates a screen listing multiple orders (for manager UI)
15	getOrderById	Order	Retrieves an order by its Id

Parameter:

- **updateStatus(newStatus: String): void**
 - **newStatus:** The updated status ("pending", "approved", "rejected", "canceled", etc.).
- **getOrderById(orderId: int): Order**
 - **orderId:** The Identifier of the order to retrieve.
 - **Return:** The Order object if found; otherwise might throw an exception.
- **calculateSubtotal(orderItems: List<OrderItem>): float**
 - **orderItems:** The list of ordered items to sum up.
 - **Return:** The subtotal of all items excluding VAT.
- **calculateVAT(subtotal: float): float**
 - subtotal: The total cost of items before VAT.
 - Return: subtotal * VAT_RATE.
- **isEligibleForFreeShipping(): boolean**
 - No parameters. Uses internal order data (subtotal, location, etc.).
 - Return: true if the order meets free shipping criteria, otherwise false.

Exception:

- **OrderNotFoundException**
 - Thrown if no order exists with the specified orderId when calling methods like getOrderById().
- **OrderAlreadyProcessedException**
 - Thrown if attempting to approve, reject, or cancel an order that has already been processed (i.e., not in "pending").

Method:

- **Order() (constructor)**
 - **Purpose:** Initializes a new order object with default values (status = "pending", createdAt = current time).
 - **Usage:** Called when the system or user creates an order (e.g., after finalizing a cart).
- **updateToReject()**
 - **Purpose:** Changes the order’s status to "rejected".
 - **Usage:** Called by a manager or system logic if an order cannot be fulfilled (out of stock, other reasons).
- **updateToApprove()**
 - **Purpose:** Changes the order’s status to "approved".
 - **Usage:** Invoked once the manager or system confirms items are in stock and the order is valid.
- **save()**
 - **Purpose:** Persists this order’s current data (status, items, totalAmount, etc.) to a database or file.
 - **Usage:** Called whenever the order state is updated (e.g., after approving or rejecting).
- **calculateSubtotal(), calculateVAT(), calculateRegularDeliveryFee(), calculateRushDeliveryFee(), calculateTotalAmount()**
 - **Purpose:** Each method breaks down the cost calculation to keep the code organized.
 - **Usage:** Typically chained together after the user finalizes or updates the order’s items or delivery method.
- **isEligibleForFreeShipping()**
 - Purpose: Checks rules (like “subtotal > 100,000 VND” or other promotional logic) to see if shipping can be waived.
 - Usage: Called during fee calculation to adjust regularDeliveryFee if free shipping applies.

State

- If status = “approved” or “rejected” → The order is considered processed and cannot be changed to another status without special rules.
- If orderItems changes → Recalculate subtotal, vatAmount, rushDeliveryFee, regularDeliveryFee, totalAmount.
- If free shipping is applicable → regularDeliveryFee might be set to 0 or partial discount.
- If paymentTransactionId is set → A corresponding payment has been recorded for this order.

28. Design Class: OrderItem

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	Int	null	Id of the product in the order
2	product	Product	null	Reference to the product object
3	quantity	int	0	Quantity of the product ordered
4	unitPrice	float	0.0	Price of the product at time of order
5	subtotal	float	0.0	Total price for this item (unitPrice × quantity)

6	isRushDeliveryEligible	boolean	false	Whether this item is eligible for rush delivery
7	weight	float	0.0	Weight of one unit of this product

Table 2 . Operation design

#	Name	Return type	Description (purpose)
1	OrderItem	vold	Constructor for creating a new OrderItem
2	calculateSubtotal	float	Calculates subtotal for this item
3	getTotalWeight	float	Calculates total weight of this item
4	checkEligibilityForRushDelivery	boolean	Checks if eligible for rush delivery
5	toOrderItemDTO	Map<String, Object>	Converts to data transfer object for display
6	getProductId	String	Returns the product Id
7	getQuantity	int	Returns the quantity
8	getUnitPrice	float	Returns the unit price
9	getSubtotal	float	Returns the subtotal
10	getProduct	Product	Returns the product object

Parameter:

- **OrderItem(product: Product, quantity: int)**
 - **product:** The complete Product object
 - **quantity:** Integer representing the quantity of this product in the order
- **OrderItem(productId: String, quantity: int, unitPrice: float, weight: float, isRushDeliveryEligible: boolean)**
 - **productId:** String representing the unique Identifier of the product
 - **quantity:** Integer representing the quantity of this product in the order
 - **unitPrice:** Float representing the price at time of order
 - **weight:** Float representing the weight of one unit
 - **isRushDeliveryEligible:** Boolean indicating if the product is eligible for rush delivery
- **calculateSubtotal(): float**
 - **Return:** Float value representing the subtotal (unitPrice × quantity)
- **getTotalWeight(): float**
 - **Return:** Float value representing the total weight (weight × quantity)
- **checkEligibilityForRushDelivery(): boolean**
 - **Return:** Boolean indicating if this item is eligible for rush delivery
- **toOrderItemDTO(): Map<String, Object>**
 - **Return:** Map containing item details for display

Exception:

- **OrderItem(product: Product, quantity: int)**
 - **NullProductException:** Thrown if product is null
 - **InvalidQuantityException:** Thrown if quantity is negative or zero
- **OrderItem(productId: String, quantity: int, unitPrice: float, weight: float, isRushDeliveryEligible: boolean)**
 - **InvalidProductIdException:** Thrown if productId is null or empty
 - **InvalidQuantityException:** Thrown if quantity is negative or zero
 - **InvalidPriceException:** Thrown if unitPrice is negative
 - **InvalidWeightException:** Thrown if weight is negative
- **getTotalWeight(): float**
 - **MissingWeightDataException:** Thrown if weight information is not available

Method:

- **OrderItem(product: Product, quantity: int)**
 - **Purpose:** Creates a new OrderItem from a Product object
 - **Processing:**
 - Valldates product and quantity
 - Extracts product details (Id, price, weight)
 - Sets isRushDeliveryEligible based on product.isSupportedPlaceRushOrder
 - Calculates initial subtotal
 - **Slide effects:** A new OrderItem is created with initialized attributes
- **OrderItem(productId: String, quantity: int, unitPrice: float, weight: float, isRushDeliveryEligible: boolean)**
 - **Purpose:** Creates a new OrderItem with specific attributes
 - **Processing:**
 - Valldates all input parameters
 - Initializes attributes with provided values
 - Calculates initial subtotal
 - **Slide effects:** A new OrderItem is created with initialized attributes
- **calculateSubtotal(): float**

- **Purpose:** Calculates the total price for this order item
 - **Processing:** Multiplies the unitPrice by the quantity
 - **Return:** The calculated subtotal
- **getTotalWeight(): float**
 - **Purpose:** Calculates the total weight of this item for shipping calculations
 - **Processing:** Multiplies the weight by the quantity
 - **Return:** The total weight in kilograms
- **checkEligibilityForRushDelivery(): boolean**
 - **Purpose:** Determines if this item is eligible for rush delivery
 - **Processing:** Returns the value of isRushDeliveryEligible attribute
 - **Return:** True if eligible for rush delivery, false otherwise
- **toOrderItemDTO(): Map<String, Object>**
 - **Purpose:** Creates a data transfer object for display or API responses
 - **Processing:**
 - Collects relevant item data into a map
 - Includes product details, quantity, price information
 - **Return:** Map containing all item details

State

- OrderItem represents a snapshot of a product at the time of order
- It stores both the product Id and optionally a reference to the complete Product object
- The unitPrice reflects the price at the time of order, which may differ from the current product price
- The subtotal is calculated and stored based on quantity and unitPrice
- The isRushDeliveryEligible flag determines whether this item can be delivered with rush service
- The weight attribute stores the product weight for shipping calculations

29. Design Class: VNPay

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	apiKey	String	null	VNPay API key
2	gatewayUrl	String	null	The VNPay endpoint (sandbox or production) for payment requests.

Table 2 . Operation design

#	Name	Return type	Description (purpose)
1	processByVNPay()	vold	Processes payments via VNPay
2	processRefund()	boolean	Processes refunds via VNPay

Parameter:

- **ProcessByVNPay(transactionInfo: PaymentTransaction)**
 - transactionInfo: Information required for VNPay payment.
- **processRefund(refundInfo: PaymentTransaction)**
 - refundInfo: Contains details about the refund (order Id, amount, etc.).

Exception

- VNPayConnectionException: If there is a failure to connect or authenticate with VNPay.
- VNPayResponseException: If VNPay returns an error response

State

- If the connection to VNPay fails → VNPayConnectionException.
- If VNPay denies the transaction → VNPayResponseException or returns false.

30. Design Class: ProductController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1				

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	CreateProduct()	vold	Create a new product with provided information
2	UpdateProduct(productId: int)	vold	Update product with updated information
3	ValldateProduct(productId: String)	boolean	Check the provided information is valld or not

Parameter:

- **UpdateProduct(productId: int)**
 - productId must be exist
- **ValldateProduct(productId: int)**
 - productId must be exist
 - The input in the form must be valld

Exception:

- **UpdateProduct(productId: int)**
 - ProductNotFoundException: If no product with the specified productId is found.
- **ValldateProduct(productId: int)**
 - ProductNotFoundException: If no product with the specified productId is found.
 - WrongFormatInputException: If input is in wrong format
 - PriceOutOfRangeException: If input price is out of range 30%-150% value

Method:

How to use parameters/attributes:

- **CreateProduct(productId: int)**
 - **Purpose:** This method creates a new product with the provided information.
 - **Usage:** It receives product details from the user or system, then creates a new product and adds it to the product list.
- **UpdateProduct(productId: int)**
 - **Purpose:** This method updates the details of an existing product based on the provided information.
 - **Usage:** If the product exists, the method updates attributes such as name, price, quantity, etc
- **ValldateProduct(productId: int)**
 - **Purpose:** This method checks whether the provided product information is valld.
 - **Usage:** It verifies required fields such as title, price, category, and quantity. If the information is valld, the method returns true; otherwise, it returns false (e.g., price out of the allowed range, negative quantity).

State:

31. Design Class: PayOrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentInvoice	Invoice	null	A reference to the invoice that is being processed for payment.
2	paymentTransaction	PaymentTransa ction	null	A reference to the PaymentTransaction created once payment is initiated or completed.

Table 2. Operation Design

#	Name	Return type	Description (purpose)
1	payOrder	boolean	Initiates payment process for an invoice and returns success/failure
2	paymentFailed	vold	Handles the scenario when the payment fails (e.g., logs, notifies user)
3	savePaymentResult	boolean	Persists the final payment result (success or failure) into the system

Parameter:

- **payOrder(invoice: Invoice): boolean**
 - **invoice:** The Invoice object to be paid
 - **Return:** true if payment initiation or completion is successful, otherwise false

Exception:

- PaymentProcessingException – Could be thrown if an error occurs during payment (e.g., network failure to VNPay, invalld invoice data, etc.).

- InvoiceNotFoundException – If the provided invoice cannot be found or is invalid.

Method:

How to use parameters/attributes:

- **payOrder(invoice: Invoice): boolean**
 - **Purpose:** To process payment for the given Invoice. It may call external boundaries (e.g., VNPayBoundary) to complete the transaction.
 - **Usage:** Called after the invoice is created and the user chooses payment method. If successful, returns true; on error, might throw PaymentProcessingException or return false.
- **paymentFailed()**
 - **Purpose:** Triggered when the payment process fails. This method might notify the user interface and log the failure.
 - **Usage:** Called by the system or boundary when a payment gateway reports an error.
- **savePaymentResult()**
 - **Purpose:** To store the payment outcome (success, failure, transaction details) into the database or system logs.
 - **Usage:** Called once the payment is confirmed or fails, ensuring the final state is recorded.

State

- If payOrder succeeds → The system moves to a “payment completed” state for the Invoice.
- If payOrder fails → paymentFailed() is invoked; the system may keep the invoice state as “unpaid,” logs the failure, etc.
- savePaymentResult() typically changes or finalizes the payment record in the database.

32. Design Class: PlaceOrderService

Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation Design

#	Name	Return type	Description (purpose)
1	checkRushDeliverySupport	boolean	check rush delivery condition

Parameter:

- checkRushDeliveryCondition(province: String, district: String, cart: Cart): boolean
 - **province, district:** the provided address information
 - **cart:** containing list of CartItem.
 - **Return:** true if payment initiation or completion is successful, otherwise false

Exception: no specific information

Method:

How to use parameters/attributes:

- checkRushDeliveryCondition(
 - **Purpose:** to verify rush delivery conditions and mark which one is eligible.
 - **Usage:** used by PlaceOrderController to check address information provided by UI and mark which CartItem is eligible.

State

- If ‘province’ is not ‘Hanoi’, return false
- If ‘province’ is Hanoi but ‘district’ is not specified, return false.
- Else iterating through CartItem in the ‘Cart’ to mark which ‘CartItem’ is eligible.